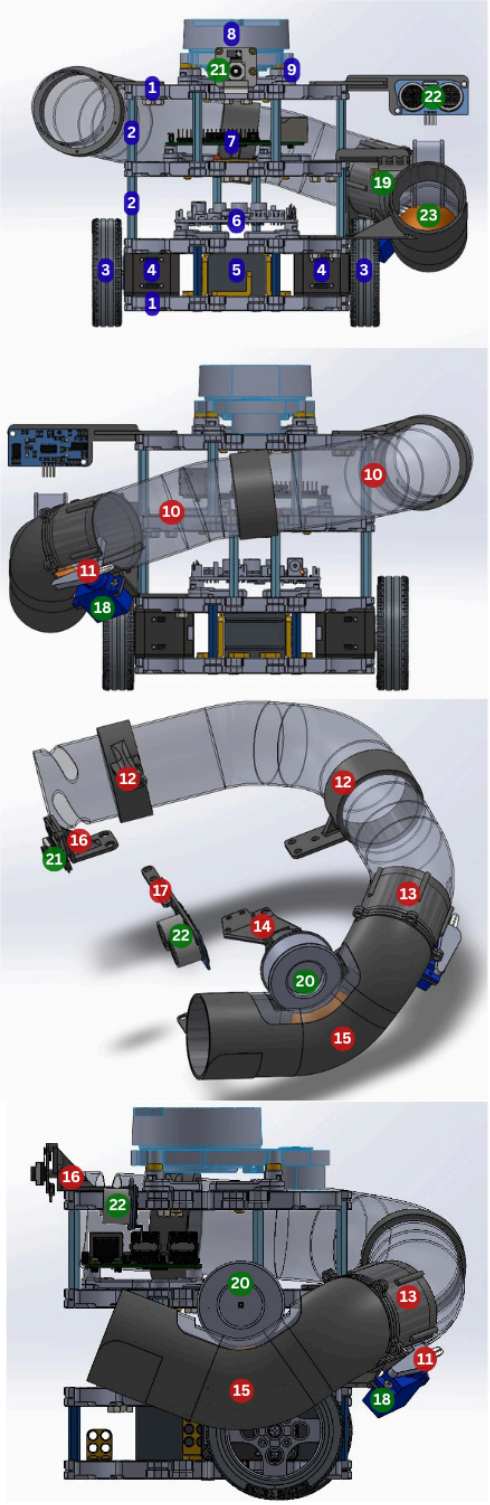
End User Documentation

1. System Overview

The system is an Autonomous Mobile Robot (AMR) based on the TurtleBot3 architecture, designed for delivery missions in dynamic, unmapped warehouse environments. The primary objective is to autonomously transport and precisely drop designated ping pong ball payloads across an unknown maze infrastructure.



1. TurtleBot chassis frame (stacked plates)
2. Plate support pillars
3. Wheel and Tire
4. 2x Dynamixel DC gear motors
5. Lipo Battery
6. Opencr1.0
7. Raspberry Pi
8. LDS
9. USB2LDS

10. Aluminium wire framework
11. Elbow bend Gate for Servo extention
12. Tube Mount clamp middle and end
13. Servo and shooting pipe Mounting bracket
14. DC motor Mounting bracket
15. Front shooting pipe
16. Rpi Camera Bracket
17. Ultrasonic sensor bracket

18. Servo motor
19. DC motor
20. Flywheel wheel
21. Rpi Camera
22. Ultrasonic sensors
23. Mission-critical payload module (PingPong)
24. M4 Hex screw and M4 Nut + M2 cross screw and M2 nut

The following table (Table 1) outlines the fundamental hardware and software specifications of the AMR. Ensure all maintenance and operational tasks respect these threshold values.

Table 1: Technical Specification and Operating Parameters

Item	Specification/ Value	Description
System module/ Platform	Turtlebot3 (Burger)	TurtleBot3 based Autonomous Mobile Robot prototype, core hardware architecture
Software	ROS2 Humble	Primary operational framework using Raspberry Pi and OpenCR as main control
Total system weight	1197 g	Dry weight including the launching mechanism
Payload Type	Ping pong balls	9 ping pong balls for maximum load
Launching system	Flywheel/ servo motor/ ultrasonic sensor	All three components are driven by 5V, and the control lines are connected to the Raspberry Pi
Battery (for Turtlebot)	11.1V and 1800 mAh	Main power source
Battery (for launching system)	1.5V and 1100 mAh	4 AA battery connected in series to provide enough power to launching system
Centre of Gravity	9.2cm from the ground	The geometry is relative to the left wheel of the turtlebot (origin)
Nominal Launch Distance	2 cm to 20 cm	The distance will vary when the battery voltage decreases

2. Pre-Mission Checklist for System and Robot

Phase 1: Mechanical & Electrical Inspection (Power OFF)

- ☐ Check all PLA brackets are secure, without cracks, deformation, or loose fasteners. Ensure the wheels rotate freely without obstruction.
- ☐ Confirm the LiDAR, camera, ultrasonic sensor, servo motor and flywheel are mounted securely, not blocked and working properly.
- ☐ Check all the wiring is secured, properly insulted, and routed safely away from the moving parts.
- ☐ Make sure the servo latch rotates properly and returns to its home position. Ensure the flywheel spins freely without rubbing nearby parts.
- ☐ Confirm the battery is connected properly, and placed securely without falling off.
- ☐ Ensure all the payload is attached properly and will not falling off

Phase 2: System Power ON & Hardware Initialization & Software Stack

- ☐ Check that the Raspberry Pi, OpenCR, and required subsystems are powered on properly (the starting sound is heard, the LEDs are turned on, and the LiDAR is spinning).
- ☐ Ensure that the flywheel is not spinning and the servo motor is in the starting position.
- ☐ Execute the main ROS launch sequence and confirm there are no fatal terminal errors and confirm that all other ROS nodes are active.
- ☐ Verify via RViz that the LiDAR point cloud and camera streams are updating correctly in real time.

3. System Guide

The system operates on a continuous SLAM and frontier-based exploration cycle, integrating 2D LiDAR scans with wheel odometry to generate a dynamic occupancy grid map. Navigation and obstacle avoidance are managed by the ROS navigation stack's dual-layer planning. When the vision subsystem identifies an ArUco marker, the robot enters 'Delivery Mode' and autonomously aligns itself with the station. The Finite State Machine (FSM) then coordinates with the OpenCR microcontroller to execute a precise delivery loop—actuating the motor to release exactly three ping pong balls. After deployment, the system resets to exploration mode to continue its mission.

4. Operating Procedures

1. **Initialization & Connection**: Perform physical checks and power on the system. Once the Raspberry Pi and OpenCR boot successfully, establish an SSH connection to the Pi via WiFi.
2. **System Launch**: Execute the core ROS launch files and start the SLAM, localization, and autonomous exploration nodes.
3. **Autonomous Mission Loop**: The AMR explores the environment, automatically detecting ArUco markers and actuating the payload delivery mechanism. This cycle repeats continuously until all objectives are met.
4. **Data Preservation & Software Halt**: Save all generated map data and diagnostic logs. Safely terminate the autonomous mission loop, ensuring all motor commands have fully stopped.
5. **Safe Power-Off & Cleanup**: Shut down the Raspberry Pi OS. Wait for the LiDAR to stop spinning and indicator lights to turn off, then power off the OpenCR, disconnect the batteries, and clear any remaining payloads.

5. Acceptable Defect Log

The following defects may be considered acceptable provided performance remains within required limits:

Table 2: Acceptable Deviation and Tolerance Table

Defect Type	Acceptable Condition	Remarks
minor 3D print surface roughness	yes	acceptable if fit and motion are unaffected
slight wire routing untidiness	yes	acceptable if wires are insulated and secured

small launcher vibration & noise	yes	acceptable if launch repeatability remains within test tolerance
minor map edge noise	yes	acceptable if navigation remains reliable and the overall map can be seen
slight marker detection delay	yes	acceptable if objective can still be completed

6. Factory Acceptance Test (FAT)

The factory acceptance test is designed to validate that the AMR system strictly adheres to the engineering specifications before deployment. The testing scope encompasses the following core categories: hardware integrity (stability of the chassis, PLA brackets, and payload mechanism), electrical connection and software integrity (proper wiring insulation and wifi connection), power battery, firmware and software readiness, and mapping and localization.

Table 3: Detail FAT Table

Test No.	Test Item	Test Method	Acceptance Criteria	Result
FAT-01	Visual & Structural Inspection	Inspect chassis, fasteners, and wiring.	No loose parts; all wiring is secured and safe.	Pass [ ] Fail [ ]
FAT-02	Power & Battery Test	Measure battery voltage (resting and under load).	Resting voltage > 10.3 V; load voltage > 9.5 V with no system resets.	Pass [ ] Fail [ ]
FAT-03	Network & Connection	Ping Raspberry Pi over WiFi.	Connection established; Packet loss < 3 %; latency < 40 ms.	Pass [ ] Fail [ ]
FAT-04	LiDAR Readiness	Launch /scan nodes and monitor data streams.	Consistent point cloud data published with no blind spots.	Pass [ ] Fail [ ]
FAT-05	Camera & Vision Test	Launch camera node and test ArUco detection.	Live image available; ArUco detected reliably at 2 meters.	Pass [ ] Fail [ ]
FAT-06	Ultrasonic Sensor	Measure obstacle distance (17–23 cm).	Reading within ± 3 cm tolerance of actual distance.	Pass [ ] Fail [ ]
FAT-07	Kinematics & Drive	Run wheel_test file (1 minute).	Travels 4 m (± 20 cm) and rotates 360° (± 10°) without stalling/drift.	Pass [ ] Fail [ ]

FAT-08	SLAM & Mapping	Teleoperate to map 5x5 m test maze.	2D occupancy grid generated successfully without major distortion.	Pass [ ] Fail [ ]
FAT-09	Path Planning & Nav	Issue a 2D Nav Goal in RViz.	Robot autonomously generates a path and reaches the target (± 5 cm) with zero collisions.	Pass [ ] Fail [ ]
FAT-10	Feeder Actuation	Command servo to cycle 3 times (reservoir loaded).	Servo completes sweep; exactly one ball released per cycle.	Pass [ ] Fail [ ]
FAT-11	Launcher Spin Test	Command flywheel to target RPM (via PWM).	Flywheel reaches and maintains target RPM; no over heat/abnormal vibrations.	Pass [ ] Fail [ ]
FAT-12	Fixed Target Launch	Align with static ArUco and trigger launch.	Payload lands within a 4 cm radius of the target receptacle.	Pass [ ] Fail [ ]

The AMR system is considered to have successfully passed the FAT and is ready for deployment when all of the following conditions are strictly met:

1. Mandatory Safety Compliance (Critical Failure)
   - Zero tolerance for safety failures: The system must achieve a "Pass" on all critical safety-related and electrical tests, specifically FAT-01, and FAT-02. Any failure in these categories constitutes an immediate FAT failure.
2. Core System Functionality
   - Navigation & Perception: The system must successfully generate a usable map and navigate autonomously without collision, passing FAT-04 to FAT-09.
   - Payload Mission Execution: meeting the quantifiable accuracy tolerances specified in FAT-10 and FAT-11
3. Defect Management & Resolution
   - No Unresolved Major Faults: There must be zero unresolved "Fail" statuses across all 12 FAT test items.
   - Permissible Defects Allowed: Minor deviations are only acceptable if they are explicitly listed and justified within Table 3, provided they do not compromise the system's structural integrity, electrical safety, or core mission objectives.
   - Sign-off: All final test logs, recorded data (e.g., SLAM maps, current draw readings), and documented defect waivers must be reviewed and signed off by the lead engineer and the acceptance authority.

7. Maintenance and Part Replacement Log

While the AMR system is built for durability, certain electro-mechanical components undergo higher stress during normal operation. The following critical parts should be monitored regularly and replaced when specific wear indicators are observed:

1. Payload servo motor:
2. Launcher DC Motor & Flywheel Assembly:
3. Primary Li-Po Battery Pack:
4. Power Harness and Terminals:

5. AA battery pack:

Date (YY/MM/DD))	Part / Component	Part ID / Description	Reason for Replacement	Replaced by	Remarks